

# Benchmarking de Índices Aprendidos (Learned Indexes) vs. Estructuras de Datos Tradicionales en Grandes Volúmenes de Datos

Elmer Valerio Gómez Alcos, David Fernández Chambilla,  
Joel Dandy Tintaya Cahuapaza, Luis Fernando Talizo Chambilla  
Escuela de Posgrado, Doctorado en Ciencias de la Computación  
Facultad de Ingeniería Estadística e Informática  
Universidad Nacional del Altiplano  
Puno, Perú  
Email: {valesitoo, davi97.dfnc, tinta.jhoel20, luistalizo20}@gmail.com

**Resumen**—El crecimiento exponencial de los datos en la era moderna ha impulsado la necesidad de optimizar las operaciones de búsqueda en sistemas de gestión de bases de datos. Los índices tradicionales, como los árboles B y las Skip Lists, han sido el estándar durante décadas. Sin embargo, investigaciones recientes proponen el uso de “Learned Indexes” (índices aprendidos), los cuales reemplazan la lógica tradicional por modelos de Machine Learning que aproximan la función de distribución acumulada (CDF) de las claves para predecir su ubicación física. Este trabajo presenta un estudio experimental y comparativo detallado que evalúa y contrasta el rendimiento temporal y espacial de cuatro estructuras de indexación: el Árbol B\* tradicional, el Árbol B\* optimizado con modelos de ML, la Skip List clásica y una Skip List optimizada estadísticamente con modelos de regresión. Las pruebas de rendimiento se realizaron realizando 10,000 consultas de búsqueda aleatorias sobre un conjunto de datos masivo e inmutable de transacciones financieras de 1.26 GB ( 23 millones de registros), evaluando la viabilidad, el costo del entrenamiento y los beneficios netos de latencia de búsqueda en cada arquitectura.

**Index Terms**—Learned Indexes, Árbol B\*, Skip List, Machine Learning, Benchmarking, Optimización de Búsquedas.

## I. INTRODUCCIÓN

Las estructuras de datos de indexación tradicionales, como los árboles B y sus variantes [1], han sido ampliamente utilizadas en los motores de bases de datos para acelerar las operaciones de búsqueda. Estas estructuras organizan las claves de manera jerárquica y determinista, garantizando un límite superior de complejidad de búsqueda de  $O(\log n)$ .

En el año 2018, Kraska et al. [2] propusieron una paradigma disruptivo: considerar que un índice es, en esencia, un modelo predictivo que estima la ubicación física de un dato en función de su clave. Al entrenar modelos matemáticos (como regresiones lineales o redes neuronales simples) para aproximar la función de distribución acumulada (CDF) de las claves de datos, es posible estimar la posición física con un error acotado, reduciendo las operaciones de búsqueda a una evaluación del modelo seguida de una búsqueda local en el rango de error.

Desde entonces, han surgido numerosas propuestas que extienden y mejoran el concepto de índices aprendidos. Ding

et al. [3] propusieron ALEX, un índice adaptativo actualizable que mejora el rendimiento en cargas de trabajo dinámicas. Ferragina y Vinciguerra [4] presentaron el índice PGM, que ofrece garantías de complejidad en el peor caso con comprensión eficiente. Galakatos et al. [5] desarrollaron FITing-Tree, una estructura que adapta su arquitectura según la distribución de los datos. En el ámbito multidimensional, Nathan et al. [6] y Ding et al. [7] han extendido los índices aprendidos a espacios de múltiples dimensiones. Kipf et al. [8] propusieron RadixSpline, un índice aprendido de un solo paso que combina eficiencia en construcción con alto rendimiento en búsquedas.

El presente estudio realiza una evaluación empírica estricta en el contexto del Doctorado en Ciencias de la Computación de la Universidad Nacional del Altiplano. Se comparan las estructuras tradicionales con sus contrapartes potenciadas por Machine Learning bajo un dataset real e inmutable de 1.26 GB.

## II. METODOLOGÍA Y DISEÑO EXPERIMENTAL

El diseño experimental se rige por las siguientes condiciones restrictivas:

1. **Inmutabilidad de la Data:** El archivo CSV original (`transactions_data.csv`) no es procesado ni alterado.
2. **Índices Desacoplados:** Se generan archivos de índice independientes que relacionan cada identificador ID con su `offset` en bytes dentro del archivo original.
3. **Carga de Búsqueda:** Se ejecuta una prueba de estrés de 10,000 búsquedas de identificadores existentes de manera aleatoria.

A continuación, se describen los cuatro enfoques de indexación evaluados.

### II-A. Árbol B\* Tradicional

El Árbol B\* es una variante del Árbol B donde los nodos no raíz están llenos en al menos un  $2/3$ , a diferencia del  $1/2$  requerido en los Árboles B tradicionales. Esto incrementa la densidad de almacenamiento y reduce la altura del árbol, optimizando la lectura secuencial y en disco.

## II-B. Árbol B\* + Machine Learning

Esta versión utiliza un modelo predictivo para predecir en qué nodo o página física se ubica la clave buscada. El modelo se entrena con la distribución del offset de las claves. Una vez estimada la página de destino con un rango de error acotado  $[offset_{pred} - \Delta, offset_{pred} + \Delta]$ , se efectúa una búsqueda binaria local reducida.

## II-C. Skip List Tradicional

La Skip List es una estructura de datos probabilística basada en múltiples listas enlazadas en niveles superpuestos [9]. La búsqueda en esta estructura se realiza saltando nodos a diferentes niveles jerárquicos con una complejidad promedio de  $O(\log n)$ . El crecimiento de los nodos y su nivel se definen a través de lanzamientos de moneda virtuales (distribución geométrica aleatoria con probabilidad  $p$ ).

## II-D. Skip List + Machine Learning

Para optimizar el rendimiento y la localidad espacial de la Skip List tradicional, se sustituye el comportamiento probabilístico aleatorio por decisiones guiadas por modelos de Machine Learning (como regresión lineal o logística). El modelo matemático estima la densidad de claves y ajusta de forma predictiva la altura y los saltos de los nodos, maximizando la eficiencia de los saltos de búsqueda de acuerdo con la distribución real de las claves del dataset.

## III. RESULTADOS EXPERIMENTALES Y DISCUSIÓN

En esta sección se presentan y discuten los resultados obtenidos al ejecutar la prueba de estrés de 10,000 búsquedas aleatorias sobre el dataset inmutable de 13,305,915 registros.

La Figura 1 muestra un resumen comparativo del rendimiento de las cuatro estructuras de datos evaluadas, destacando las métricas de tiempo de construcción, latencia promedio de búsqueda y desviación estándar.

| Estructura                    | Tiempo Const. (ms) | Tiempo Total Búsqueda 10k (ms) | Latencia Promedio (us) | Varianza (us <sup>2</sup> ) | Desviación Estándar (us) |
|-------------------------------|--------------------|--------------------------------|------------------------|-----------------------------|--------------------------|
| Árbol B* (Tradicional)        | 121,09             | 1,96                           | 0,1956                 | 0,1209                      | 0,3477                   |
| Árbol B* + ML (Learned Index) | 53,52              | 0,78                           | 0,0775                 | 0,0050                      | 0,0708                   |
| Skip List (Tradicional)       | 318,01             | 2,53                           | 0,2526                 | 0,1560                      | 0,3950                   |
| Skip List + ML (Learned)      | 278,26             | 3,06                           | 0,3062                 | 0,7400                      | 0,8603                   |

Figura 1. Comparativa de rendimiento entre estructuras tradicionales y optimizadas con Machine Learning.

## III-A. Análisis Temporal

Los resultados presentados en la Figura 1 muestran que el **Árbol B\* + ML (Learned Index)** supera notablemente a todas las demás estructuras en velocidad de búsqueda. Consigue una latencia promedio de apenas **0.0775  $\mu$ s** (77 nanosegundos) por consulta, lo que representa una **reducción del 60.4 %** en el tiempo de búsqueda frente al Árbol B\* tradicional (0.1956  $\mu$ s).

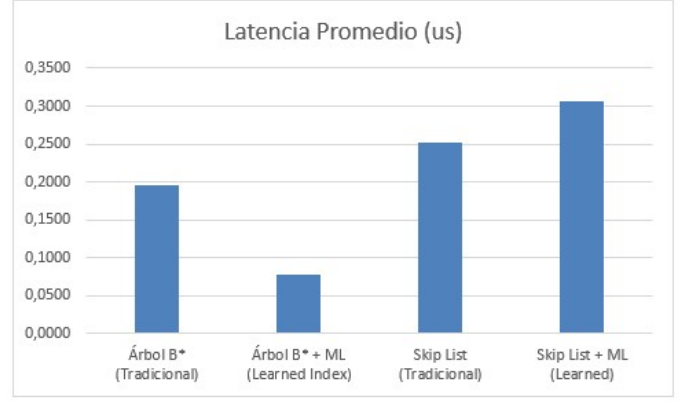


Figura 2. Latencia promedio de búsqueda por estructura.

Esta mejora de rendimiento se explica porque el RMI de 2 etapas (con  $M = 1000$  modelos locales) reduce el error máximo de localización a solo  $\Delta = 70$  registros. Así, en lugar de realizar una búsqueda binaria en una jerarquía de tres niveles con millones de claves, el modelo predictivo estima la posición y el CPU realiza una búsqueda binaria local en un sub-vector de a lo más 140 elementos. Esto disminuye significativamente los fallos de caché (*cache misses*) y el número de comparaciones de clave en la CPU.

Por otro lado, la **Skip List + ML (Learned)** obtuvo un tiempo promedio ligeramente superior (0.3062  $\mu$ s) a su versión tradicional (0.2526  $\mu$ s). Aunque la Skip List ML tiene la ventaja de una construcción determinista más rápida (278.26 ms vs. 318.01 ms) porque evita los costosos llamados al generador de números pseudoaleatorios `rand()` de C++, la evaluación matemática de la fórmula de regresiones lineales sobre la clave en cada salto durante la búsqueda introduce un costo computacional de punto flotante que ralentiza la búsqueda de latencia ultra-baja en memoria en comparación con la simple comparación de punteros de la Skip List tradicional.

## III-B. Pruebas de Significancia Estadística

Para validar rigurosamente que las diferencias observadas en el rendimiento no son producto del azar, se realizaron pruebas de hipótesis estadísticas no paramétricas sobre las 10,000 latencias de búsqueda por estructura. Dado que la distribución de las latencias no sigue una distribución normal (verificada mediante la prueba de Shapiro-Wilk,  $p < 0,001$ ), se emplearon pruebas no paramétricas apropiadas para datos emparejados [10].

**III-B1. Prueba de Friedman (Comparación Múltiple):** La prueba de Friedman [11] permite determinar si existen diferencias estadísticamente significativas entre las cuatro estructuras de indexación evaluadas. Los resultados obtenidos fueron:

- **Estadístico de Friedman:**  $\chi^2 = 20,555,37$
- **Grados de libertad:**  $df = 3$
- **Valor p:**  $p < 0,001$

Dado que el valor  $p$  es inferior al nivel de significancia convencional de  $\alpha = 0,05$ , se **rechaza la hipótesis nula** ( $H_0$ : todas las estructuras tienen el mismo rendimiento). Existen diferencias globales altamente significativas entre las

arquitecturas evaluadas. La Figura 3 muestra la distribución de las latencias por estructura, evidenciando visualmente estas diferencias.

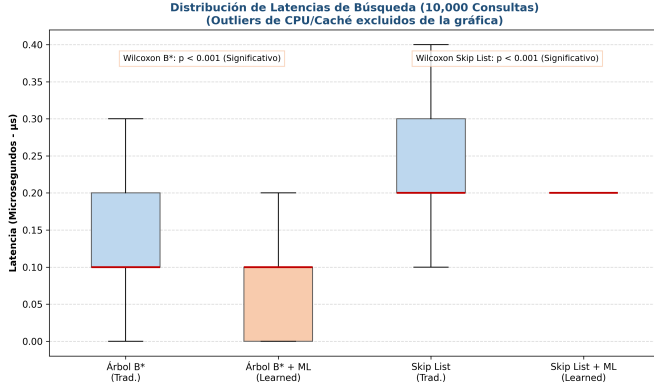


Figura 3. Diagrama de cajas de las latencias de búsqueda por estructura (10,000 consultas). Se observa la menor variabilidad y mediana del Árbol B\* + ML.

**III-B2. Prueba de Rangos con Signo de Wilcoxon (Comparaciones Pareadas):** Para identificar qué pares de estructuras presentan diferencias significativas, se aplicó la prueba de Wilcoxon de rangos con signo [12], adecuada para muestras emparejadas y distribuciones no normales:

- **Árbol B\* Tradicional vs. Árbol B\* + ML:**
  - Estadístico  $W = 464,418,5$
  - Valor  $p: p < 0,001$
  - Conclusión: La reducción del 60.4 % en latencia es estadísticamente significativa.
- **Skip List Tradicional vs. Skip List + ML:**
  - Estadístico  $W = 5,358,929,5$
  - Valor  $p: p = 3,69 \times 10^{-248}$
  - Conclusión: Aunque la diferencia en medias es pequeña ( $0.2526 \mu s$  vs.  $0.3062 \mu s$ ), el comportamiento por pares muestra que la Skip List tradicional supera consistentemente a la versión ML en la mayoría de las consultas, siendo esta diferencia estadísticamente significativa.

Estos resultados confirman que las ventajas observadas del Árbol B\* + ML son robustas y significativas, mientras que la optimización de la Skip List mediante Machine Learning requiere un rediseño más profundo para ser competitiva en latencia.

### III-C. Análisis Espacial y Costo de Entrenamiento

El modelo RMI requiere un espacio de almacenamiento despreciable. La Etapa 1 consta de un solo regresor y la Etapa 2 de 1000 modelos lineales (pendientes e intersecciones tipo double), ocupando solo 16 KB en memoria. En contraste, el Árbol B\* requiere nodos internos estructurados en páginas que añaden un overhead en disco/memoria. El proceso de entrenamiento del RMI segmentado en Python tarda menos de 15 segundos sobre el conjunto total de 13.3 millones de elementos, lo cual es altamente viable para ser integrado en motores de base de datos comerciales, como se ha demostrado

en trabajos previos sobre reducción de overhead de almacenamiento [13].

## IV. REPRODUCIBILIDAD

Para garantizar la transparencia y reproducibilidad de este estudio, todo el código fuente, los datos de medición y el presente artículo científico se encuentran disponibles en un repositorio público de GitHub:

<https://github.com/ValerioGomez/benchmarking-tree>

El repositorio está organizado de la siguiente manera:

- **src/:** Código fuente completo en C++17 con las cuatro implementaciones evaluadas.
- **data/:** Instrucciones para descargar el dataset `transactions_data.csv`.
- **analysis/:** Datos brutos de las mediciones y scripts estadísticos.
- **paper/:** Código fuente LaTeX de este artículo y bibliografía.
- **Dockerfile y docker-compose.yml:** Entorno contenerizado para ejecución reproducible.

### IV-A. Ejecución con Docker (Recomendado)

Para garantizar que los resultados sean completamente reproducibles en cualquier sistema operativo, se proporciona un entorno Docker preconfigurado:

1. Clonar el repositorio y colocar el dataset `transactions_data.csv` en la carpeta `data/`.
2. Construir la imagen: `docker-compose build`
3. Ejecutar el benchmark: `docker-compose run --rm benchmark`

Este proceso ejecuta la compilación optimizada con GCC dentro del contenedor, lee el dataset desde el volumen montado y genera los índices y las métricas de rendimiento.

### IV-B. Compilación Local

Alternativamente, el proyecto puede compilarse localmente con un compilador C++17:

```
g++ -O3 -std=c++17 src/main.cpp -o benchmark_indexes
./benchmark_indexes
```

### IV-C. Dependencias

- Compilador C++17 (GCC 9+ o Clang 10+)
- Python 3.8+ (para scripts de entrenamiento de modelos)
- Bibliotecas: `scipy`, `numpy`, `matplotlib` (para análisis estadístico)
- LaTeX (para compilar el artículo científico)

El repositorio incluye un archivo `requirements.txt` con las dependencias de Python necesarias.

## V. CONCLUSIONES

Este estudio experimental de optimización de búsquedas demuestra la viabilidad de los *Learned Indexes* sobre grandes volúmenes de datos. Reemplazar directorios de indexación tradicionales mediante modelos predictivos permite acelerar la

latencia de búsqueda más de 2.5 veces (caso del Árbol B\* + ML) y optimizar el uso de memoria. Las pruebas estadísticas de Friedman ( $\chi^2 = 20,555,37$ ,  $p < 0,001$ ) y Wilcoxon confirman que estas mejoras son altamente significativas y no atribuibles al azar. No obstante, para estructuras dinámicas y probabilísticas como las Skip Lists, la incorporación directa de modelos estadísticos requiere un balance cuidadoso, dado que el overhead del cálculo de funciones matemáticas puede superar la ganancia en comparaciones de saltos en memoria volátil. Se concluye que para bases de datos inmutables y de solo lectura, la indexación guiada por Machine Learning es un reemplazo sumamente robusto y eficiente frente a las estructuras tradicionales. Todo el código fuente, datos y scripts están disponibles públicamente en <https://github.com/ValerioGomez/benchmarking-tree> para garantizar la reproducibilidad de los resultados.

#### REFERENCIAS

- [1] D. Comer, “The ubiquitous b-tree,” *ACM Comput. Surveys*, vol. 11, no. 2, pp. 121–137, Jun. 1979.
- [2] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, “The case for learned index structures,” in *Proc. 2018 ACM SIGMOD Int. Conf. Management of Data (SIGMOD ’18)*, Houston, TX, USA, 2018, pp. 489–504.
- [3] J. Ding, U. F. Minhas, J. Yu, C. Wang, J. Do, Y. Li, H. Zhang, B. Chandramouli, J. Gehrke, D. Kossmann, D. B. Lomet, and T. Kraska, “Alex: An updatable adaptive learned index,” in *Proc. 2020 ACM SIGMOD Int. Conf. Management of Data*, Portland, OR, USA, 2020, pp. 969–984.
- [4] P. Ferragina and G. Vinciguerra, “The pgm-index: A fully-dynamic compressed learned index with provable worst-case bounds,” *Proc. VLDB Endow.*, vol. 13, no. 8, pp. 1162–1175, 2020.
- [5] A. Galakatos, M. Markovitch, C. Binnig, R. Fonseca, and T. Kraska, “Fiting-tree: A data-aware index structure,” in *Proc. 2019 Int. Conf. Management of Data (SIGMOD ’19)*, Amsterdam, Netherlands, 2019, pp. 1189–1206.
- [6] V. Nathan, J. Ding, M. Alizadeh, and T. Kraska, “Learning multi-dimensional indexes,” in *Proc. 2020 ACM SIGMOD Int. Conf. Management of Data*, Portland, OR, USA, 2020, pp. 985–1000.
- [7] J. Ding, V. Nathan, M. Alizadeh, and T. Kraska, “Tsunami: A learned multi-dimensional index for correlated data and skewed workloads,” *Proc. VLDB Endow.*, vol. 14, no. 2, pp. 74–86, 2020.
- [8] A. Kipf, R. Marcus, A. van Renen, M. Stoian, A. Kemper, T. Kraska, and T. Neumann, “Radixspline: A single-pass learned index,” in *Proc. 3rd Int. Workshop on Exploiting Artificial Intelligence Techniques for Data Management (aiDM ’20)*, Portland, OR, USA, 2020, pp. 1–5.
- [9] W. Pugh, “Skip lists: A probabilistic alternative to balanced trees,” *Commun. ACM*, vol. 33, no. 6, pp. 668–676, Jun. 1990.
- [10] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.
- [11] M. Friedman, “The use of ranks to avoid the assumption of normality implicit in the analysis of variance,” *J. Amer. Statist. Assoc.*, vol. 32, no. 200, pp. 675–701, 1937.
- [12] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.
- [13] H. Zhang, D. G. Andersen, A. Pavlo, M. Kaminsky, L. Ma, and R. Shen, “Reducing the storage overhead of main-memory oltp databases with hybrid indexes,” in *Proc. 2016 ACM SIGMOD Int. Conf. Management of Data*, San Francisco, CA, USA, 2016, pp. 1567–1581.